

Conversación 1:

Pregunta 1:

Cómo puedo adaptar este sistema de envío de packets con tcp a udp?

```
void ServerPacketTypesManager::SendData(sf::TcpSocket& socket, sf::Packet& packet)
{
    if (socket.send(packet) == sf::Socket::Status::Done) {
        std::cout << "Enviando paquete..." << std::endl;
    }
    else {
        std::cerr << "Error al enviar el paquete" << std::endl;
    }
}

void ServerPacketTypesManager::SendUdpData(sf::UdpSocket& socket, sf::Packet& packet)
{
    if (serverSocket.send(buffer, bufferSize, SERVER_IP, SERVER_PORT) == sf::Socket::Status::Done)
    }
```

Respuesta 1:

En UDP no envías directamente un `sf::Packet` como en TCP. En SFML, debes serializar el `sf::Packet` a un buffer de bytes (`const void* + tamaño`) y luego usar `sf::UdpSocket::send`.

Adaptación correcta

```
void ServerPacketTypesManager::SendUdpData(sf::UdpSocket& socket, sf::Packet& packet)
{
    const void* data = packet.getData();
    std::size_t size = packet.getDataSize();

    sf::IpAddress ip = SERVER_IP;
    unsigned short port = SERVER_PORT;

    sf::Socket::Status status = socket.send(data, size, ip, port);

    if (status == sf::Socket::Status::Done)
    {
        std::cout << "Paquete UDP enviado..." << std::endl;
    }
    else
    {
        std::cerr << "Error al enviar paquete UDP" << std::endl;
    }
}
```

Diferencias clave respecto a TCP

- TCP (`sf::TcpSocket`) → stream fiable, SFML permite `socket.send(packet)` directamente.
- UDP (`sf::UdpSocket`) → datagramas, necesitas:
 - `packet.getData()`
 - `packet.getDataSize()`
 - `socket.send(void*, size, ip, port)`

Importante en UDP

- No hay conexión persistente.
- Cada envío necesita IP y puerto.
- No hay garantía de entrega ni orden.
- El receptor debe usar `receive()` con buffer:

```
char buffer[1024];
std::size_t received;
sf::IpAddress sender;
unsigned short port;

socket.receive(buffer, sizeof(buffer), received, sender, port);
```

Si quieres, puedo ayudarte a convertir todo tu sistema TCP → UDP (incluyendo reconexión, ACKs o fiabilidad básica).

Pregunta 2

Adapta también el recibir packets:

Ejemplo de recibir udp:

```
if (socket.receive(buffer, sizeof(buffer), receivedSize, senderIp, senderPort) == sf::Socket::Status::Done)
{
    std::cout << "Mensaje recibido de " << senderIp.value() << ":" << senderPort << std::endl;

    int receivedData;
    std::memcpy(&receivedData, buffer, sizeof(receivedData));

    std::cout << "Dato recibido: " << receivedData << std::endl;
}
```

Función de tcp:

```
void NetworkManager::ReceiveClientPacket()
{
    if (!selector.isReady(listener)) {
        for (int i = 0; i < clients.size(); i++) {
            if (selector.isReady(*clients[i])) {
                sf::Packet packet;

                if (clients[i]->receive(packet) == sf::Socket::Status::Done) {
```

```

        SPTM->ReceivePacket(packet, *clients[i]);
    }
}
}
}

```

Respuesta 2:

En UDP no existe `sf::TcpSocket` ni sockets persistentes en el selector de la misma forma. Normalmente recibes directamente desde un `sf::UdpSocket` y obtienes también IP y puerto del emisor.

Versión equivalente en UDP

```

void NetworkManager::ReceiveClientPacket(sf::UdpSocket& socket)
{
    char buffer[1024];
    std::size_t receivedSize = 0;
    sf::IpAddress senderIp;
    unsigned short senderPort;

    sf::Socket::Status status =
        socket.receive(buffer, sizeof(buffer), receivedSize, senderIp, senderPort);

    if (status == sf::Socket::Status::Done)
    {
        std::cout << "Paquete recibido de "
                    << senderIp.toString()
                    << "." << senderPort << std::endl;

        sf::Packet packet;
        packet.append(buffer, receivedSize);

        SPTM->ReceivePacket(packet, senderIp, senderPort);
    }
}

```

Cambio importante respecto a TCP

Tu versión TCP:

- usa `sf::TcpSocket* clients[]`
- cada cliente es un socket persistente
- puedes usar `sf::SocketSelector`

En UDP:

- **no hay lista de clientes**
- solo hay **un socket**

- el cliente se identifica por:
 - `sf::IpAddress`
 - `unsigned short port`
-

Adaptación del “modelo cliente” (equivalente TCP → UDP)

Antes (TCP):

```
Receive(socket_cliente_i);
```

Ahora (UDP):

```
Receive(socket_global);  
+ identificar cliente por senderIp/senderPort
```

Versión más parecida a tu arquitectura TCP (recomendado)

Si quieres mantener estructura tipo “clientes”, debes mapearlos manualmente:

```
std::map<std::pair<sf::IpAddress, unsigned short>, ClientData>
```

Y en recepción:

```
auto key = std::make_pair(senderIp, senderPort);
```

Resumen del cambio conceptual

- TCP: 1 socket = 1 cliente
- UDP: 1 socket = N clientes (identificados por IP/puerto)

Si quieres, puedo adaptarte tu `SocketSelector` + `lista de clientes` a un sistema UDP completo con gestión de clientes estilo “session layer”.

Conversacion 2:

Pregunta 1:

Ahora queremos leer esta informacion del xml para pasarla por un paquete TCP como hemos hecho con los otros.
Como deberiamos hacer la parte de enviar el paquete?

```
<?xml version="1.0" encoding="utf-8"?>  
<map>
```

```

    <background>
        <position x="0" y="0"/>
        <sprite>sprites/background.png</sprite>
        <size width="1920" height="1080"/>
    </background>

    <platform>
        <position x="0" y="550"/>
        <sprite>sprites/gorund.png</sprite>
        <size width="800" height="50"/>
    </platform>
    <platform>
        <position x="300" y="250"/>
        <sprite>sprites/platform.png</sprite>
        <size width="200" height="50"/>
    </platform>

    <spawns>
        <player id="1">
            <position x="100" y="300"/>
        </player>
        <player id="2">
            <position x="700" y="100"/>
        </player>
    </spawns>
</map>

```

XMLReader.cpp:

```
#include "XMLReader.h"
```

```

XMLReader::XMLReader() {
    rapidxml::xml_document<> doc;

    std::ifstream file("gameConfig.xml");
    if (!file.is_open()) {
        std::cerr << "Error: No se pudo abrir el archivo " << std::endl;
        return;
    }

    std::stringstream buffer;
    buffer << file.rdbuf();
    file.close();

    std::string content(buffer.str());

    try {
        doc.parse<0>(&content[0]);
    }
    catch (const rapidxml::parse_error& e) {
        std::cerr << "Error de parseo XML: " << e.what() << std::endl;
        return;
    }

    rapidxml::xml_node<>* rootNode = doc.first_node();

```

```

    if (!rootNode) {
        std::cerr << "Error: Nodo raíz no encontrado." << std::endl;
        return;
    }

    //Leer informacion plataformas
    //Leer informacion fondo
    //Leer informacion jugadores
}

```

XMLReader.h:

```

#pragma once
#include "dep/inc/xml/rapidxml.hpp"
#include "dep/inc/xml/rapidxml_print.hpp"
#include "dep/inc/xml/rapidxml_utils.hpp"
#include "dep/inc/xml/rapidxml_iterators.hpp"

```

```

#include <sstream>
#include <iostream>

```

```

class XMLReader
{
    XMLReader() = default;
    XMLReader();

    ~XMLReader();
};

```

ServerPacketType.cpp:

```

#include "ServerPacketTypeManager.h"
#include "MatchmakingManager.h"
#include <iostream>

```

```

sf::Packet& operator>>(sf::Packet& packet, PacketTypes& tipo) {
    int temp;
    packet >> temp;
    tipo = static_cast<PacketTypes>(temp);

    return packet;
}

```

```

sf::Packet& operator<<(sf::Packet& packet, PacketTypes& tipo) {
    int temp;
    temp = static_cast<int>(tipo);
    packet << temp;

    return packet;
}

```

```

void ServerPacketTypesManager::ReceivePacket(sf::Packet packet, sf::TcpSocket& client)
{
    PacketTypes packetType;

    packet >> packetType;
}

```

```

switch (packetType)
{
case PacketTypes::HANDSHAKE:
    ReceiveHandshakePacket(packet);
    break;
case PacketTypes::LOGIN:
    ReceiveLoginPacket(packet, client);
    break;
case PacketTypes::REGISTER:
    ReceiveRegisterPacket(packet, client);
    break;
case PacketTypes::LOBBY_CREATE:
    ReceiveLobbyCreatePacket(packet, client);
    break;
case PacketTypes::LOBBY_JOIN:
    ReceiveLobbyJoinPacket(packet, client);
    break;
case PacketTypes::RANKING:
    ReceiveRankingPacket(packet, client);
    break;
case PacketTypes::START_GAME:
    ReceiveStartGamePacket(packet);
    break;
case PacketTypes::END_GAME:
    ReceiveEndGamePacket(packet);
    break;
default:
    std::cout << "No se ha identificado el tipo de paquete" << std::endl;
    break;
}

packet.clear();
}

void ServerPacketTypesManager::SendData(sf::TcpSocket& client, sf::Packet& packet)
{
    if (client.send(packet) == sf::Socket::Status::Done) {
        std::cout << "Enviando paquete..." << std::endl;
    }
    else {
        std::cerr << "Error al enviar el paquete" << std::endl;
    }
}

void ServerPacketTypesManager::SendHandshake(sf::TcpSocket& client)
{
    sf::Packet packet;
    packet << PacketTypes::HANDSHAKE << handshakeMessage;
    SendData(client, packet);
}

void ServerPacketTypesManager::SendUpdatedPlayerCount(sf::TcpSocket& client, int playerCount)
{

```

```

        sf::Packet packet;
        packet << PacketTypes::WAITING_ROOM_PLAYERS;
        packet << playerCount;

        SendData(client, packet);
    }

void ServerPacketTypesManager::SendLoginResponse(sf::TcpSocket& client, bool success, const std::string&
username)
{
    sf::Packet packet;
    packet << PacketTypes::LOGIN;
    packet << success;
    packet << username;

    SendData(client, packet);

    std::cout << "Respuesta de login enviada" << std::endl;
}

void ServerPacketTypesManager::SendRegisterResponse(sf::TcpSocket& client, bool success, const std::string&
username)
{
    sf::Packet packet;
    packet << PacketTypes::REGISTER;
    packet << success;
    packet << username;

    SendData(client, packet);

    std::cout << "Respuesta de registro enviada" << std::endl;
}

void ServerPacketTypesManager::SendLobbyCreateResponse(sf::TcpSocket& client, bool success)
{
    sf::Packet packet;
    packet << PacketTypes::LOBBY_CREATE;
    packet << success;

    SendData(client, packet);

    std::cout << "Respuesta de creacion de lobby enviada" << std::endl;
}

void ServerPacketTypesManager::SendLobbyJoinResponse(sf::TcpSocket& client, bool success)
{
    sf::Packet packet;
    packet << PacketTypes::LOBBY_JOIN;
    packet << success;

    SendData(client, packet);

    std::cout << "Respuesta de join a lobby enviada" << std::endl;
}

```



```

void ServerPacketTypesManager::SendRankingPacket(sf::TcpSocket& client, std::vector<Database::RankingEntry>&
rankings)
{
    sf::Packet packet;
    packet << PacketTypes::RANKING;

    packet << static_cast<int>(rankings.size());

    for (const auto& entry : rankings) {
        packet << entry.position;
        packet << entry.userId;
        packet << entry.username;
        packet << entry.points;

        std::cout << entry.position << " " << entry.userId << " - "
            << entry.username << " - "
            << entry.points << " puntos" << std::endl;
    }

    SendData(client, packet);
    std::cout << "Ranking packet enviado con " << rankings.size() << " entradas." << std::endl;
}

void ServerPacketTypesManager::SendMapPacket(sf::TcpSocket& client)
{
    sf::Packet packet;
    packet << PacketTypes::MAP;

}

void ServerPacketTypesManager::ReceiveHandshakePacket(sf::Packet data)
{
    std::string receiveMessage;
    data >> receiveMessage;

    std::cout << "Mensaje enviado del cliente: " << receiveMessage << std::endl;
}

void ServerPacketTypesManager::ReceiveLoginPacket(sf::Packet data, sf::TcpSocket& client)
{
    std::string loginUsername;
    std::string loginPassword;

    data >> loginUsername;
    data >> loginPassword;

    int userId = 0;

    bool correctLogin = DB->LoginUser(loginUsername, loginPassword, userId);

    SendLoginResponse(client, correctLogin, loginUsername);
}

```

```

        // Si es correcto, guardar también los datos del usuario (nombre y puntos del ranking)

        if (correctLogin) {
            MM->AddConnectedPlayer(&client, loginUsername, 15);
        }
    }

void ServerPacketTypesManager::ReceiveRegisterPacket(sf::Packet data, sf::TcpSocket& client)
{
    std::string registerUsername;
    std::string registerPassword;

    data >> registerUsername;
    data >> registerPassword;

    std::string passwordHash = bcrypt::generateHash(registerPassword);

    bool correctRegister = DB->RegisterUser(registerUsername, passwordHash);

    SendRegisterResponse(client, correctRegister, registerUsername);

    if (correctRegister) {
        MM->AddConnectedPlayer(&client, registerUsername, 15);
    }
}

void ServerPacketTypesManager::ReceiveLobbyCreatePacket(sf::Packet data, sf::TcpSocket& client)
{
    std::string lobbyID;

    data >> lobbyID;

    bool successfulLobbyCreation = MM->CreateWaitingRoom(lobbyID, &client);

    if (successfulLobbyCreation) {
        std::cout << "Lobby " << lobbyID << " creado exitosamente, pasando jugador a la sala de espera"
<< std::endl;
    }
    else {
        std::cout << "El ID " << lobbyID << " ya esta en uso" << std::endl;
    }

    SendLobbyCreateResponse(client, successfulLobbyCreation);
}

void ServerPacketTypesManager::ReceiveLobbyJoinPacket(sf::Packet data, sf::TcpSocket& client)
{
    std::string lobbyID;

    data >> lobbyID;

    std::cout << "Servidor recibe join packet" << std::endl;
    bool successfulLobbyJoin = MM->JoinWaitingRoom(lobbyID, &client);
}

```

```

        if (successfulLobbyJoin) {
            std::cout << "Jugador se ha unido a lobby con ID: " << lobbyID << std::endl;
        }
        else {
            std::cout << "El lobby " << lobbyID << " esta lleno o no existe" << std::endl;
        }

        SendLobbyJoinResponse(client, successfulLobbyJoin);
    }

void ServerPacketTypesManager::ReceiveRankingPacket(sf::Packet data, sf::TcpSocket& client)
{
    int playerId;

    data >> playerId;

    std::cout << "Recibida peticion de ranking para el jugador ID: " << playerId << std::endl;

    std::vector<Database::RankingEntry> topRankings = DB->GetTop10Rankings(playerId);

    SendRankingPacket(client, topRankings);
}

void ServerPacketTypesManager::ReceiveStartGamePacket(sf::Packet data)
{
}

void ServerPacketTypesManager::ReceiveEndGamePacket(sf::Packet data)
{
}

```

ServerPacketType.h:

```

#pragma once
#include <SFML/Network.hpp>
#include "Bcrypt/bcrypt.h"
#include "Database.h"

#include <string>

#define SPTM ServerPacketTypesManager::Instance()

enum PacketTypes
{
    HANDSHAKE, LOGIN, REGISTER, LOBBY_CREATE, LOBBY_JOIN, WAITING_ROOM_PLAYERS,
    RANKING, START_GAME, END_GAME, MAP
};

class ServerPacketTypesManager
{
public:
    static ServerPacketTypesManager* Instance() {
        static ServerPacketTypesManager ptm;
    }
}

```

```

        return &ptm;
    }

private:
    std::string handshakeMessage = "Handshake realizado";

public:
    void ReceivePacket(sf::Packet packet, sf::TcpSocket& client);
    void SendHandshake(sf::TcpSocket& client);
    void SendUpdatedPlayerCount(sf::TcpSocket& client, int playerCount);

private:
    ServerPacketTypesManager() = default;
    ServerPacketTypesManager(const ServerPacketTypesManager& ptm) = delete;
    ServerPacketTypesManager& operator=(const ServerPacketTypesManager& ptm) = delete;
    ~ServerPacketTypesManager() = default;

    void SendData(sf::TcpSocket& client, sf::Packet& packet);

    void SendLoginResponse(sf::TcpSocket& client, bool success, const std::string& message);
    void SendRegisterResponse(sf::TcpSocket& client, bool success, const std::string& message);
    void SendLobbyCreateResponse(sf::TcpSocket& client, bool success);
    void SendLobbyJoinResponse(sf::TcpSocket& client, bool success);
    void SendRankingPacket(sf::TcpSocket& client, std::vector<Database::RankingEntry>& rankings);
    void SendMapPacket(sf::TcpSocket& client);

    void ReceiveHandshakePacket(sf::Packet data);
    void ReceiveLoginPacket(sf::Packet data, sf::TcpSocket& client);
    void ReceiveRegisterPacket(sf::Packet data, sf::TcpSocket& client);
    void ReceiveLobbyCreatePacket(sf::Packet data, sf::TcpSocket& client);
    void ReceiveLobbyJoinPacket(sf::Packet data, sf::TcpSocket& client);
    void ReceiveRankingPacket(sf::Packet data, sf::TcpSocket& client);
    void ReceiveStartGamePacket(sf::Packet data);
    void ReceiveEndGamePacket(sf::Packet data);
};

```

Respuesta 1:

Para enviar el contenido del XML a través de TCP, tienes que **serializar los datos del mapa en el paquete** de manera estructurada, no enviar el XML crudo (aunque también podrías, pero es menos eficiente).

Aquí te muestro cómo implementar `SendMapPacket()` y cómo modificar `XMLReader` para que pueda ser usado tanto por servidor como por cliente.

1. Modificar XMLReader para extraer y almacenar datos

Primero, completa `XMLReader` para que almacene los datos del mapa y pueda proporcionarlos:

```

**XMLReader.h:**
```cpp

```

```

#pragma once
#include "dep/inc/xml/rapidxml.hpp"
#include "dep/inc/xml/rapidxml_print.hpp"
#include "dep/inc/xml/rapidxml_utils.hpp"
#include "dep/inc/xml/rapidxml_iterators.hpp"

#include <sstream>
#include <iostream>
#include <vector>
#include <string>

struct Position {
 float x, y;
};

struct Size {
 float width, height;
};

struct Background {
 Position position;
 std::string sprite;
 Size size;
};

struct Platform {
 Position position;
 std::string sprite;
 Size size;
};

struct SpawnPoint {
 int playerId;
 Position position;
};

class XMLReader {
public:
 XMLReader(const std::string& filename);
 ~XMLReader();

 // Getters
 const Background& getBackground() const { return m_background; }
 const std::vector<Platform>& getPlatforms() const { return m_platforms; }
 const std::vector<SpawnPoint>& getSpawns() const { return m_spawns; }

private:
 Background m_background;
 std::vector<Platform> m_platforms;
 std::vector<SpawnPoint> m_spawns;
};
...

```

**\*\*XMLReader.cpp:\*\***

```

```cpp
#include "XMLReader.h"

XMLReader::XMLReader(const std::string& filename) {
    rapidxml::xml_document<> doc;

    std::ifstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Error: No se pudo abrir el archivo " << filename << std::endl;
        return;
    }

    std::stringstream buffer;
    buffer << file.rdbuf();
    file.close();

    std::string content(buffer.str());

    try {
        doc.parse<0>(&content[0]);
    }
    catch (const rapidxml::parse_error& e) {
        std::cerr << "Error de parseo XML: " << e.what() << std::endl;
        return;
    }

    rapidxml::xml_node<>* rootNode = doc.first_node("map");
    if (!rootNode) {
        std::cerr << "Error: Nodo raíz 'map' no encontrado." << std::endl;
        return;
    }

    // Leer background
    rapidxml::xml_node<>* bgNode = rootNode->first_node("background");
    if (bgNode) {
        rapidxml::xml_node<>* posNode = bgNode->first_node("position");
        if (posNode) {
            m_background.position.x = std::stof(posNode->first_attribute("x")->value());
            m_background.position.y = std::stof(posNode->first_attribute("y")->value());
        }

        rapidxml::xml_node<>* spriteNode = bgNode->first_node("sprite");
        if (spriteNode) {
            m_background.sprite = spriteNode->value();
        }

        rapidxml::xml_node<>* sizeNode = bgNode->first_node("size");
        if (sizeNode) {
            m_background.size.width = std::stof(sizeNode->first_attribute("width")->value());
            m_background.size.height = std::stof(sizeNode->first_attribute("height")->value());
        }
    }

    // Leer plataformas

```

```

for (rapidxml::xml_node<>* platformNode = rootNode->first_node("platform");
    platformNode;
    platformNode = platformNode->next_sibling("platform")) {

    Platform platform;

    rapidxml::xml_node<>* posNode = platformNode->first_node("position");
    if (posNode) {
        platform.position.x = std::stof(posNode->first_attribute("x")->value());
        platform.position.y = std::stof(posNode->first_attribute("y")->value());
    }

    rapidxml::xml_node<>* spriteNode = platformNode->first_node("sprite");
    if (spriteNode) {
        platform.sprite = spriteNode->value();
    }

    rapidxml::xml_node<>* sizeNode = platformNode->first_node("size");
    if (sizeNode) {
        platform.size.width = std::stof(sizeNode->first_attribute("width")->value());
        platform.size.height = std::stof(sizeNode->first_attribute("height")->value());
    }

    m_platforms.push_back(platform);
}

// Leer spawns
rapidxml::xml_node<>* spawnsNode = rootNode->first_node("spawns");
if (spawnsNode) {
    for (rapidxml::xml_node<>* playerNode = spawnsNode->first_node("player");
        playerNode;
        playerNode = playerNode->next_sibling("player")) {

        SpawnPoint spawn;
        spawn.playerId = std::stoi(playerNode->first_attribute("id")->value());

        rapidxml::xml_node<>* posNode = playerNode->first_node("position");
        if (posNode) {
            spawn.position.x = std::stof(posNode->first_attribute("x")->value());
            spawn.position.y = std::stof(posNode->first_attribute("y")->value());
        }

        m_spawns.push_back(spawn);
    }
}

XMLReader::~XMLReader() {
}
...

---
```

2. Implementar SendMapPacket en ServerPacketTypeManager

****ServerPacketTypeManager.cpp - Añadir:****

```
```cpp
void ServerPacketTypesManager::SendMapPacket(sf::TcpSocket& client)
{
 sf::Packet packet;
 packet << PacketTypes::MAP;

 // Cargar el XML
 XMLReader mapReader("gameConfig.xml");

 // Enviar background
 const Background& bg = mapReader.getBackground();
 packet << bg.position.x << bg.position.y;
 packet << bg.sprite;
 packet << bg.size.width << bg.size.height;

 // Enviar número de plataformas y cada plataforma
 const auto& platforms = mapReader.getPlatforms();
 packet << static_cast<int>(platforms.size());
 for (const auto& platform : platforms) {
 packet << platform.position.x << platform.position.y;
 packet << platform.sprite;
 packet << platform.size.width << platform.size.height;
 }

 // Enviar número de spawns y cada spawn
 const auto& spawns = mapReader.getSpawns();
 packet << static_cast<int>(spawns.size());
 for (const auto& spawn : spawns) {
 packet << spawn.playerId;
 packet << spawn.position.x << spawn.position.y;
 }

 SendData(client, packet);
 std::cout << "Mapa enviado al cliente" << std::endl;
}
```
```

3. Recibir el mapa en el cliente

En el cliente necesitas una función similar para recibir y reconstruir el mapa:

```
```cpp
void ClientPacketManager::ReceiveMapPacket(sf::Packet& packet)
{
 Background bg;
 packet >> bg.position.x >> bg.position.y;
 packet >> bg.sprite;
 packet >> bg.size.width >> bg.size.height;

 int platformCount;
```



```

packet >> platformCount;
std::vector<Platform> platforms;
for (int i = 0; i < platformCount; i++) {
 Platform p;
 packet >> p.position.x >> p.position.y;
 packet >> p.sprite;
 packet >> p.size.width >> p.size.height;
 platforms.push_back(p);
}

int spawnCount;
packet >> spawnCount;
std::vector<SpawnPoint> spawns;
for (int i = 0; i < spawnCount; i++) {
 SpawnPoint s;
 packet >> s.playerId;
 packet >> s.position.x >> s.position.y;
 spawns.push_back(s);
}

// Usar los datos para inicializar el nivel
InitializeLevel(bg, platforms, spawns);
}
...

```

---

#### ## 4. Resumen del flujo

1. **Servidor** → `SendMapPacket(client)` se llama cuando el juego está por empezar
2. El servidor lee `gameConfig.xml` con `XMLReader`
3. Los datos se serializan en el `sf::Packet` en orden: `MAP` tipo → background → platforms → spawns
4. El cliente recibe el paquete, identifica `PacketTypes::MAP` y llama a `ReceiveMapPacket()`
5. El cliente reconstruye el nivel con los datos recibidos

---

#### ## 5. Alternativa más simple (enviar XML crudo)

Si prefieres no modificar la serialización, puedes enviar el XML como string:

```

...cpp
void ServerPacketTypesManager::SendMapPacket(sf::TcpSocket& client)
{
 sf::Packet packet;
 packet << PacketTypes::MAP;

 std::ifstream file("gameConfig.xml");
 std::stringstream buffer;
 buffer << file.rdbuf();
 std::string xmlContent = buffer.str();

 packet << xmlContent;
 SendData(client, packet);
}

```

}  
...

El cliente lo recibe y parsea con `XMLReader` directamente. Es más fácil pero menos eficiente si el mapa es grande.

**\*\*¿Qué opción prefieres?\*\*** La primera es más profesional y eficiente, la segunda más rápida de implementar.

# Conversación 3

## Pregunta 1:

Dentro del MovementInterpolation como lo deberiamos hacer para que interpole la posicion del Character Oponent en base a la información recibida del servidor almacenada en MovementPacket? La interpolación debe ser enter los dos último paquetes que sea con la posicion actual del oponente y la posición del paquete por orden en la lista, si el ID del siguiente paquete es menor que el del ultimo paquete lo ignoramos.

Movement Interpolation.h:

```
#pragma once
#include <map>
#include <vector>
#include <algorithm>
#include "User.h"
#include "MovementPacket.h"
#include "Vector2.h"

class MovementInterpolation
{
private:
 std::map<User, std::vector<MovementPacket>> packetList;
 std::map<User, int> currentMovementPacketIDList;

public:
 MovementInterpolation();
};
```

MovementPacket.h:

```
#pragma once
#include <SFML/Network/Packet.hpp>
#include "Vector2.h"

struct MovementPacket
{
public:
 int ID;
 Vector2 pos;
};

inline sf::Packet& operator <<(sf::Packet& packet, const Vector2& vector)
{
 return packet << vector.x << vector.y;
}

inline sf::Packet& operator >>(sf::Packet& packet, Vector2& vector)
{
 return packet >> vector.x >> vector.y;
}

inline sf::Packet& operator <<(sf::Packet& packet, const MovementPacket& movement)
{
}
```

```

 return packet << movement.ID << movement.pos;
 }

 inline sf::Packet& operator >>(sf::Packet& packet, MovementPacket& movement)
 {
 return packet >> movement.ID >> movement.pos;
 }

```

```

User.h:
#pragma once
#include <string>
struct User
{
 std::string nickname;
 int score;
 int userIndex;
 int position;
 float speed;
};

```

```

GameScene.h:
#pragma once

```

```

#include <SFML/Graphics.hpp>
#include <optional>

```

```

#include "NetworkManager.h"
#include "MovementPrediction.h"
#include "MovementInterpolation.h"
#include "PlayerCharacter.h"

```

```

class GameScene
{
private:
 MovementPrediction movementPrediction;

 sf::RectangleShape ground;
 sf::Clock clock;

 PlayerCharacter player;
 Character oponent;
public:
 GameScene()
 : ground({ 800.f, 50.f })
 {
 ground.setFillColor(sf::Color(100, 100, 100));
 ground.setPosition({ 0.f, 550.f });
 }

 bool Init()
 {
 NT->Init();

 if (NT->GetDisconnectFromServer())

```

```

 return false;

 return true;
}

void HandleEvents(sf::RenderWindow& window)
{
 while (const std::optional event = window.pollEvent())
 {
 if (event->is<sf::Event::Closed>())
 window.close();

 if (const auto* keyPressed = event->getIf<sf::Event::KeyPressed>())
 {
 if (keyPressed->scancode == sf::Keyboard::Scancode::Space)
 player.ResetJumpBuffer();
 }
 }
}

void Update()
{
 float dt = clock.restart().asSeconds();
 float groundY = ground.getPosition().y;

 player.Update(dt, groundY);
 oponent.Update(dt, groundY);

 if (NT->GetDisconnectFromServer()) return;

 NT->Update();

 if (movementPrediction.ShouldSendPacket(dt))
 {
 MovementPacket playerMovementPacket =
movementPrediction.CreateMovementPacket(player.GetPosition());
 NT->SendMovementPacket(playerMovementPacket);
 }
}

void Render(sf::RenderWindow& window)
{
 window.clear(sf::Color(40, 40, 40));

 window.draw(ground);
 window.draw(player.GetSprite());
 window.draw(oponent.GetSprite());

 window.display();
}
};

```

Cahracter.cpp:  
 #include "Character.h"

```

void Character::HandleAnimation(float dt)
{
 if (isQuacking)
 {
 sprite.Update(dt);

 if (sprite.IsCurrentAnimationFinished())
 {
 isQuacking = false;
 sprite.StartAnimation("idle", true);
 }

 return;
 }

 bool isMoving = std::abs(velocity.x) > 20.f;

 if (isMoving)
 sprite.StartAnimation("walk");
 else
 sprite.StartAnimation("idle");

 if (facingRight)
 sprite.FlipHorizontally(false);
 else
 sprite.FlipHorizontally(true);

 sprite.Update(dt);
}

Character::Character(const std::string& texturePath) : sprite(texturePath), quackSound(quackBuffer)
{
 position = { 400.f, 300.f };
 sprite.SetScale({ scale, scale });

 sprite.AddAnimation("idle", Animation(32, 32, 0, 1, 0));
 sprite.AddAnimation("walk", Animation(32, 32, 1, 6, 0.1f));
 sprite.AddAnimation("shoot", Animation(32, 32, 2, 5, 0.05f, false));
 sprite.AddAnimation("quack", Animation(32, 32, 3, 1, 0.5f, false));

 sprite.StartAnimation("idle");

 velocity = sf::Vector2f(0, 0);

 width = sprite.GetWidth() * scale;
 height = sprite.GetHeight() * scale;

 if (!quackBuffer.loadFromFile("assets/quack.wav"))
 std::cout << "Error cargando quack.wav" << std::endl;

 quackSound.setVolume(20.f);
}

```

```

Character.h:
#pragma once
#include <SFML/Graphics.hpp>
#include <SFML/Audio.hpp>
#include <iostream>
#include "AnimatedSprite.h"

class Character
{
protected:
 sf::SoundBuffer quackBuffer;
 sf::Sound quackSound;

 const float scale = 4.f;
 float width;
 float height;

 AnimatedSprite sprite;

 sf::Vector2f position;
 sf::Vector2f velocity;

 const float jumpBufferTime = 0.12f;
 float jumpBufferTimer = 0.f;

 bool isOnGround = false;

 bool facingRight = true;

 bool isQuacking = false;

 void HandleAnimation(float dt);

public:
 Character(const std::string& texturePath = "assets/grey_duck.png");

 sf::Sprite GetSprite()
 {
 return sprite.GetSprite();
 }

 sf::Vector2f GetPosition()
 {
 return position;
 }

 void ResetJumpBuffer()
 {
 jumpBufferTimer = jumpBufferTime;
 }

 virtual void Update(float dt, float groundY)
 {
 HandleAnimation(dt);
 }

```

```

 sprite.SetPosition(position);
 }
};

```

MovementValidation.h:

```

#pragma once
#include <map>
#include "MovementPacket.h"
#include <vector>
#include "User.h"

```

```

std::vector<User> gameUsers = { {"eauna", 0, 0, 0, 1.0f} };

```

```

class MovementValidation

```

```

{

```

```

private:

```

```

 std::map<User, std::vector<MovementPacket>> packetList;
 float validationTolerance = 0.5f;

```

```

public:

```

```

 std::map<User, MovementPacket> ValidatePackets() {
 std::map<User, MovementPacket> validatedPackets;
 for (User user : gameUsers) {
 validatedPackets[user] = ValidateUserPackets(user);
 packetList[user].clear();
 packetList[user].push_back(validatedPackets[user]);
 }
 }

```

```

 return validatedPackets;

```

```

}

```

```

void AddPacket(User user, MovementPacket packet) { packetList[user].push_back(packet); }

```

```

private:

```

```

 MovementPacket ValidateUserPackets(User user) {
 MovementPacket resultPacket = packetList[user][0];
 for (int i = 1; i < packetList[user].size(); ++i) {

```

```

 if (packetList[user][i].ID < resultPacket.ID) continue;

```

```

 int physicsMult = packetList[user][i].ID - resultPacket.ID;

```

```

 //Early exit to error

```

```

 if (std::abs((packetList[user][i].pos - resultPacket.pos).SqrMagnitude() - physicsMult *
physicsMult * user.speed * user.speed)

```

```

 > validationTolerance)
 return resultPacket;

```

```

 resultPacket = packetList[user][i];

```

```

 }

```

```

 }

```

```

};

```



## Respuesta 1:

Basándome en tu código, te muestro cómo implementar la interpolación de movimiento para el oponente en `MovementInterpolation`:

## MovementInterpolation.h (actualizado)

```
```cpp
#pragma once
#include <map>
#include <vector>
#include <algorithm>
#include "User.h"
#include "MovementPacket.h"
#include "Vector2.h"

class MovementInterpolation
{
private:
    std::map<User, std::vector<MovementPacket>>> packetList;
    std::map<User, int> currentMovementPacketIDList;

    // Nuevo: almacena la posición interpolada actual de cada usuario
    std::map<User, Vector2> interpolatedPositions;

    // Nuevo: tiempo de interpolación entre paquetes
    float interpolationTime = 0.f;
    float interpolationDuration = 0.1f; // Duración de interpolación en segundos

public:
    MovementInterpolation();

    // Añadir nuevo paquete recibido del servidor
    void AddPacket(const User& user, const MovementPacket& packet);

    // Actualizar la interpolación (llamar cada frame con delta time)
    void Update(float dt);

    // Obtener la posición interpolada actual de un usuario
    Vector2 GetInterpolatedPosition(const User& user) const;

    // Verificar si hay interpolación activa para un usuario
    bool HasActiveInterpolation(const User& user) const;
};
```
```

## MovementInterpolation.cpp

```
```cpp
#include "MovementInterpolation.h"

MovementInterpolation::MovementInterpolation()
{
    interpolationTime = 0.f;
}
```

```

    interpolationDuration = 0.1f;
}

void MovementInterpolation::AddPacket(const User& user, const MovementPacket& packet)
{
    // Verificar si tenemos paquetes previos para este usuario
    if (packetList.find(user) == packetList.end())
    {
        packetList[user] = std::vector<MovementPacket>();
        currentMovementPacketIDList[user] = -1;
        interpolatedPositions[user] = packet.pos;
    }

    auto& packets = packetList[user];

    // Si el ID del nuevo paquete es menor que el último procesado, lo ignoramos
    if (packet.ID <= currentMovementPacketIDList[user])
        return;

    // Añadir el paquete a la lista
    packets.push_back(packet);

    // Mantener solo los últimos 2 paquetes para interpolación
    if (packets.size() > 2)
        packets.erase(packets.begin());

    // Si tenemos al menos 2 paquetes, reiniciamos el tiempo de interpolación
    if (packets.size() >= 2)
    {
        interpolationTime = 0.f;
        // Ordenar por ID para asegurar que están en orden correcto
        std::sort(packets.begin(), packets.end(),
            [](const MovementPacket& a, const MovementPacket& b) {
                return a.ID < b.ID;
            });
    }
}

void MovementInterpolation::Update(float dt)
{
    for (auto& userPackets : packetList)
    {
        const User& user = userPackets.first;
        auto& packets = userPackets.second;

        if (packets.size() >= 2)
        {
            // Avanzar el tiempo de interpolación
            interpolationTime += dt;

            // Calcular factor de interpolación (0 a 1)
            float t = interpolationTime / interpolationDuration;

            if (t >= 1.f)

```

```

{
    // Interpolación completada, usar el último paquete
    interpolatedPositions[user] = packets.back().pos;
    currentMovementPacketIDList[user] = packets.back().ID;

    // Eliminar el paquete más antiguo
    if (packets.size() > 1)
        packets.erase(packets.begin());
}
else
{
    // Interpolación entre los dos últimos paquetes
    const MovementPacket& prevPacket = packets[packets.size() - 2];
    const MovementPacket& currentPacket = packets.back();

    // Verificar que los IDs estén en orden
    if (currentPacket.ID > prevPacket.ID)
    {
        // Interpolación lineal
        Vector2 interpolatedPos;
        interpolatedPos.x = prevPacket.pos.x * (1 - t) + currentPacket.pos.x * t;
        interpolatedPos.y = prevPacket.pos.y * (1 - t) + currentPacket.pos.y * t;

        interpolatedPositions[user] = interpolatedPos;
    }
    else
    {
        // IDs fuera de orden, usar la posición actual
        interpolatedPositions[user] = packets.back().pos;
    }
}
}
else if (!packets.empty())
{
    // Solo tenemos un paquete, usar esa posición directamente
    interpolatedPositions[user] = packets.back().pos;
    currentMovementPacketIDList[user] = packets.back().ID;
}
}
}

```

```

Vector2 MovementInterpolation::GetInterpolatedPosition(const User& user) const
{
    auto it = interpolatedPositions.find(user);
    if (it != interpolatedPositions.end())
        return it->second;

    return Vector2{0.f, 0.f}; // Posición por defecto
}

```

```

bool MovementInterpolation::HasActiveInterpolation(const User& user) const
{
    auto it = packetList.find(user);
    if (it != packetList.end())

```

```

        return it->second.size() >= 2;
    return false;
}
...

```

GameScene.h actualizado

```

...cpp
#pragma once

#include <SFML/Graphics.hpp>
#include <optional>

#include "NetworkManager.h"
#include "MovementPrediction.h"
#include "MovementInterpolation.h"
#include "PlayerCharacter.h"

class GameScene
{
private:
    MovementPrediction movementPrediction;
    MovementInterpolation movementInterpolation; // Añadir esto

    sf::RectangleShape ground;
    sf::Clock clock;

    PlayerCharacter player;
    Character oponent;

    // Para almacenar el usuario oponente
    User opponentUser;

public:
    GameScene()
        : ground({ 800.f, 50.f })
    {
        ground.setFillColor(sf::Color(100, 100, 100));
        ground.setPosition({ 0.f, 550.f });

        // Configurar usuario oponente
        opponentUser.nickname = "Opponent";
        opponentUser.score = 0;
        opponentUser.userIndex = 1;
        opponentUser.position = 0;
        opponentUser.speed = 200.f; // Velocidad base
    }

    bool Init()
    {
        NT->Init();

        if (NT->GetDisconnectFromServer())
            return false;
    }
}

```

```

    return true;
}

void HandleEvents(sf::RenderWindow& window)
{
    while (const std::optional event = window.pollEvent())
    {
        if (event->is<sf::Event::Closed>())
            window.close();

        if (const auto* keyPressed = event->getIf<sf::Event::KeyPressed>())
        {
            if (keyPressed->scancode == sf::Keyboard::Scancode::Space)
                player.ResetJumpBuffer();
        }
    }
}

void Update()
{
    float dt = clock.restart().asSeconds();
    float groundY = ground.getPosition().y;

    player.Update(dt, groundY);

    // Actualizar interpolación del oponente
    movementInterpolation.Update(dt);

    // Obtener posición interpolada y actualizar oponente
    Vector2 interpolatedPos = movementInterpolation.GetInterpolatedPosition(opponentUser);
    opponent.SetPosition(sf::Vector2f(interpolatedPos.x, interpolatedPos.y));
    opponent.Update(dt, groundY);

    if (NT->GetDisconnectFromServer()) return;

    NT->Update();

    if (movementPrediction.ShouldSendPacket(dt))
    {
        MovementPacket playerMovementPacket =
movementPrediction.CreateMovementPacket(player.GetPosition());
        NT->SendMovementPacket(playerMovementPacket);
    }

    // Procesar paquetes recibidos del oponente (esto debería venir del NetworkManager)
    ProcessReceivedPackets();
}

void Render(sf::RenderWindow& window)
{
    window.clear(sf::Color(40, 40, 40));

    window.draw(ground);
}

```

```

        window.draw(player.GetSprite());
        window.draw(oponent.GetSprite());

        window.display();
    }

private:
    void ProcessReceivedPackets()
    {
        // Aquí procesas los paquetes recibidos del servidor para el oponente
        // Por ejemplo:
        /*
        while (HayPaquetesDelOponente())
        {
            MovementPacket packet = ObtenerSiguientePaquete();
            movementInterpolation.AddPacket(opponentUser, packet);
        }
        */
    }
};
...

```

Character.h actualizado

Añade este método a la clase `Character`:

```

...cpp
public:
    // ... métodos existentes ...

    void SetPosition(const sf::Vector2f& newPosition)
    {
        position = newPosition;
    }
...

```

Uso básico:

1. ****Cuando recibes un paquete del servidor**** para el oponente, llamas a:

```

...cpp
movementInterpolation.AddPacket(opponentUser, receivedPacket);
...

```
2. ****En cada frame****, llamas a `movementInterpolation.Update(dt)` para actualizar la interpolación
3. ****Obtienes la posición interpolada**** y actualizas el sprite del oponente:

```

...cpp
Vector2 interpolatedPos = movementInterpolation.GetInterpolatedPosition(opponentUser);
oponent.SetPosition(sf::Vector2f(interpolatedPos.x, interpolatedPos.y));
...

```

Características clave:

- ****Interpola entre los últimos 2 paquetes**** válidos

- ****Ignora paquetes con ID menor**** al último procesado
- ****Mantiene el orden cronológico**** ordenando por ID
- ****Transición suave**** usando interpolación lineal (lerp)
- ****Gestión automática**** del tiempo de interpolación

La interpolación crea movimiento fluido entre las posiciones recibidas del servidor, eliminando el "stuttering" que ocurriría si aplicarlas las posiciones directamente.

Conversación 4:

Pregunta 1:

Cómo puedo hacer que los hilos esperen de forma más eficiente?

```
#pragma once
#include <vector>
#include <thread>
#include "Task.h"
#include <mutex>
#include <deque>

#define THREAD_COUNT 100
#define ThrdM ThreadManager::Instance()

class ThreadManager
{
private:
    std::vector<std::thread> threads;
    std::deque<Task*> tasks;
    std::mutex tasksMutex;

public:
    static ThreadManager* Instance() {
        static ThreadManager tm;

        return &tm;
    }

    void Init() {
        for (int i = 0; i < THREAD_COUNT; ++i) threads.emplace_back(Worker);
    }

    void AddTask(Task* task) {
        tasksMutex.lock();
        tasks.push_back(task);
        tasksMutex.unlock();
    }

    void AddUrgentTask(Task* task) {
        tasksMutex.lock();
        tasks.push_front(task);
        tasksMutex.unlock();
    }

private:
    void Worker() {
        while (true) {

            Task* task = nullptr;
```



```

        tasksMutex.lock();
        if (!tasks.empty()) {
            task = tasks.front();
            tasks.pop_front();
        }
        tasksMutex.unlock();

        if(task) task->Invoke();
    }
}
};

```

Respuesta 1:

Ahora mismo tus hilos están en *busy waiting* (spin continuo): si no hay tareas, vuelven a comprobar inmediatamente el **deque**, consumiendo CPU innecesariamente. La forma eficiente es usar **bloqueo con condition_variable** para dormir los hilos hasta que haya trabajo.

Problema actual

```

while (true) {
    lock
    comprobar cola
    unlock
    si no hay tarea -> bucle inmediato (100 hilos consumiendo CPU)
}

```

Solución: **std::condition_variable**

Permite:

- Dormir hilos cuando no hay tareas
- Despertarlos solo cuando se añade trabajo

Versión corregida

```

#pragma once
#include <vector>
#include <thread>
#include "Task.h"
#include <mutex>
#include <deque>
#include <condition_variable>

#define THREAD_COUNT 100
#define ThrdM ThreadManager::Instance()

class ThreadManager
{
private:
    std::vector<std::thread> threads;
    std::deque<Task*> tasks;

    std::mutex tasksMutex;

```

```

std::condition_variable cv;

bool stop = false;

public:
    static ThreadManager* Instance() {
        static ThreadManager tm;
        return &tm;
    }

    void Init() {
        for (int i = 0; i < THREAD_COUNT; ++i)
            threads.emplace_back(&ThreadManager::Worker, this);
    }

    void AddTask(Task* task) {
        {
            std::lock_guard<std::mutex> lock(tasksMutex);
            tasks.push_back(task);
        }
        cv.notify_one();
    }

    void AddUrgentTask(Task* task) {
        {
            std::lock_guard<std::mutex> lock(tasksMutex);
            tasks.push_front(task);
        }
        cv.notify_one();
    }

private:
    void Worker() {
        while (true) {
            Task* task = nullptr;

            {
                std::unique_lock<std::mutex> lock(tasksMutex);

                cv.wait(lock, [&] {
                    return stop || !tasks.empty();
                });

                if (stop && tasks.empty())
                    return;

                task = tasks.front();
                tasks.pop_front();
            }

            task->Invoke();
        }
    }
}

```

```
};
```

Qué mejora esto

Antes

- 100 hilos en loop constante
- CPU desperdiciada
- latencia irregular bajo carga

Ahora

- hilos dormidos cuando no hay trabajo
- wake-up inmediato con `notify_one()`
- escalabilidad real tipo thread pool

Mejora adicional (recomendación importante)

100 hilos suele ser excesivo. Regla general:

```
threads ≈ hardware_concurrency()
int count = std::thread::hardware_concurrency();
```

Extra (nivel más avanzado)

Si quieres rendimiento aún mayor:

- usar **work stealing deque**
- o **lock-free queue (moodycamel, folly)**
- o separar prioridades (colas múltiples)

Si quieres, puedo ayudarte a convertir esto en un **thread pool moderno con futures** (`std::future`), que es el estándar en C++ actual.

Conversación 5:

Pregunta 1:

Olvida todo el rapidxml. Queremos cambiarlo por un archivo de texto conformado solo por caracteres al estilo de:

```
#####  
#           #  
#   ###   #  
#           #  
# P       O #  
#####
```

Con la idea general de esto:

```
enum class Tiles {
```

```
    FLOOR = '#',
```

```
    SPAWN = 'S',
```

```
    ...
```

```
}
```

```
void HandleTile(char tile, int posx, int posy) {
```

```
    switch(tile) {
```

```
        case Tiles::FLOOR:
```

```
            map->tiles.Add(new Floor(posx, posy));
```

```
            break;
```

```
        case Tiles::SPAWN:
```

```
            map->tiles.Add(new Spawn(posx, posy));
```

```
            break;
```

```
        ...
```

```
    }
```

```
}
```

```
#include "XMLReader.h"
```

```
//Codigo generado por IA
```

```
XMLReader::XMLReader(const std::string& filename) {
```

```
    rapidxml::xml_document<> doc;
```

```
    std::ifstream file(filename);
```

```
    if (!file.is_open()) {
```

```
        std::cerr << "Error: No se pudo abrir el archivo " << filename << std::endl;
```

```
        return;
```

```
    }
```

```
    std::stringstream buffer;
```

```
    buffer << file.rdbuf();
```

```
    file.close();
```

```
    std::string content(buffer.str());
```

```
    try {
```

```
        doc.parse<0>(&content[0]);
```

```

}
catch (const rapidxml::parse_error& e) {
    std::cerr << "Error de parseo XML: " << e.what() << std::endl;
    return;
}

rapidxml::xml_node<>* rootNode = doc.first_node(NODE_MAP);
if (!rootNode) {
    std::cerr << "Error: Nodo raíz 'map' no encontrado." << std::endl;
    return;
}

rapidxml::xml_node<>* bgNode = rootNode->first_node(NODE_BACKGROUND);
if (bgNode) {
    rapidxml::xml_node<>* posNode = bgNode->first_node(NODE_POSITION);
    if (posNode) {
        m_background.position.x = std::stof(posNode->first_attribute(ATTRIBUTE_POSITION_X)->value());
        m_background.position.y = std::stof(posNode->first_attribute(ATTRIBUTE_POSITION_Y)->value());
    }

    rapidxml::xml_node<>* spriteNode = bgNode->first_node(NODE_SPRITE);
    if (spriteNode) {
        m_background.sprite = spriteNode->value();
    }

    rapidxml::xml_node<>* sizeNode = bgNode->first_node(NODE_SIZE);
    if (sizeNode) {
        m_background.size.width = std::stof(sizeNode->first_attribute(ATTRIBUTE_SIZE_WIDTH)->value());
        m_background.size.height = std::stof(sizeNode->first_attribute(ATTRIBUTE_SIZE_HEIGHT)->value());
    }
}

for (rapidxml::xml_node<>* platformNode = rootNode->first_node(NODE_PLATFORM);
    platformNode;
    platformNode = platformNode->next_sibling(NODE_PLATFORM)) {

    Platform platform;

    rapidxml::xml_node<>* posNode = platformNode->first_node(NODE_POSITION);
    if (posNode) {
        platform.position.x = std::stof(posNode->first_attribute(ATTRIBUTE_POSITION_X)->value());
        platform.position.y = std::stof(posNode->first_attribute(ATTRIBUTE_POSITION_Y)->value());
    }

    rapidxml::xml_node<>* spriteNode = platformNode->first_node(NODE_SPRITE);
    if (spriteNode) {
        platform.sprite = spriteNode->value();
    }

    rapidxml::xml_node<>* sizeNode = platformNode->first_node(NODE_SIZE);
    if (sizeNode) {
        platform.size.width = std::stof(sizeNode->first_attribute(ATTRIBUTE_SIZE_WIDTH)->value());
        platform.size.height = std::stof(sizeNode->first_attribute(ATTRIBUTE_SIZE_HEIGHT)->value());
    }
}

```

```

        m_platforms.push_back(platform);
    }

    rapidxml::xml_node<>* spawnsNode = rootNode->first_node(NODE_SPAWNS);
    if (spawnsNode) {
        for (rapidxml::xml_node<>* playerNode = spawnsNode->first_node(NODE_PLAYER);
            playerNode;
            playerNode = playerNode->next_sibling(NODE_PLAYER)) {

            SpawnPoint spawn;
            spawn.playerId = std::stoi(playerNode->first_attribute(NODE_PLAYER_ID)->value());

            rapidxml::xml_node<>* posNode = playerNode->first_node(NODE_POSITION);
            if (posNode) {
                spawn.position.x = std::stof(posNode->first_attribute(ATTRIBUTE_POSITION_X)->value());
                spawn.position.y = std::stof(posNode->first_attribute(ATTRIBUTE_POSITION_Y)->value());
            }

            m_spawns.push_back(spawn);
        }
    }
}

```

```

#pragma once
#include "dep/inc/xml/rapidxml.hpp"
#include "dep/inc/xml/rapidxml_print.hpp"
#include "dep/inc/xml/rapidxml_utils.hpp"
#include "dep/inc/xml/rapidxml_iterators.hpp"

```

```

#include <sstream>
#include <iostream>

```

```

#define NODE_MAP "map"
#define NODE_BACKGROUND "background"
#define NODE_POSITION "position"
#define ATTRIBUTE_POSITION_X "x"
#define ATTRIBUTE_POSITION_Y "y"
#define NODE_SPRITE "sprite"
#define NODE_SIZE "size"
#define ATTRIBUTE_SIZE_WIDTH "width"
#define ATTRIBUTE_SIZE_HEIGHT "height"
#define NODE_SPAWNS "spawns"
#define NODE_PLATFORM "platform"
#define NODE_PLAYER "player"
#define NODE_PLAYER_ID "id"

```

```

struct Position {
    float x, y;
};

```

```

struct Size {
    float width, height;
};

```

```

struct Background {
    Position position;
    std::string sprite;
    Size size;
};

struct Platform {
    Position position;
    std::string sprite;
    Size size;
};

struct SpawnPoint {
    int playerId;
    Position position;
};

class XMLReader
{
public:
    XMLReader(const std::string& filename);
    ~XMLReader() = default;

    inline Background GetBackground() { return m_background; }
    inline std::vector<Platform> GetPlatforms() { return m_platforms; }
    inline std::vector<SpawnPoint> GetSpawnPoints() { return m_spawns; }

private:
    Background m_background;
    std::vector<Platform> m_platforms;
    std::vector<SpawnPoint> m_spawns;
};

```

Respuesta 1:

¡Excelente decisión! Los mapas basados en caracteres son mucho más simples y directos. Vamos a crear un nuevo sistema que reemplace completamente `XMLReader` con un lector de mapas de texto.

Nuevo diseño: `MapReader` para archivos de texto

1. Archivo de cabecera: `MapReader.h`

```

```cpp
#pragma once
#include <string>
#include <vector>
#include <fstream>
#include <iostream>
#include <algorithm>

// Tipos de tiles posibles

```

```

enum class TileType {
 FLOOR = '#', // Suelo/pared
 SPAWN = 'S', // Punto de aparición del jugador
 ENEMY_SPAWN = 'E', // Punto de aparición de enemigos
 POWERUP = 'P', // Power-up
 OBJECTIVE = 'O', // Objetivo/portal
 PLATFORM = '=', // Plataforma (diferente del suelo)
 LADDER = 'H', // Escalera (vertical)
 SPIKE = '^', // Trampa
 EMPTY = ' ' // Espacio vacío
};

// Estructura para almacenar cada tile con su posición
struct Tile {
 TileType type;
 int x, y; // Posición en la cuadrícula (columna, fila)
 char originalChar; // Carácter original para debug

 Tile(TileType t, int col, int row, char c)
 : type(t), x(col), y(row), originalChar(c) {}
};

// Clase principal para manejar el mapa
class MapReader {
public:
 MapReader() = default;

 // Cargar mapa desde archivo de texto
 bool LoadFromFile(const std::string& filename);

 // Getters
 const std::vector<std::vector<Tile*>>& GetGrid() const { return m_grid; }
 const std::vector<Tile*>& GetTiles() const { return m_tiles; }
 int GetWidth() const { return m_width; }
 int GetHeight() const { return m_height; }

 // Obtener tiles por tipo
 std::vector<Tile*> GetTilesByType(TileType type) const;

 // Obtener spawn del jugador (primer 'S' encontrado)
 Tile* GetPlayerSpawn() const;

 // Obtener todos los spawns de enemigos
 std::vector<Tile*> GetEnemySpawns() const;

 // Limpiar memoria
 void Clear();

 ~MapReader() { Clear(); }

private:
 std::vector<std::vector<Tile*>> m_grid; // Matriz 2D de punteros a tiles
 std::vector<Tile*> m_tiles; // Lista plana de todos los tiles
 int m_width = 0;

```



```

int m_height = 0;

void ProcessTile(char tileChar, int col, int row);
};
...

```

### 2. Archivo de implementación: `MapReader.cpp`

```

...cpp
#include "MapReader.h"

bool MapReader::LoadFromFile(const std::string& filename) {
 std::ifstream file(filename);
 if (!file.is_open()) {
 std::cerr << "Error: No se pudo abrir el archivo " << filename << std::endl;
 return false;
 }

 Clear();

 std::vector<std::string> lines;
 std::string line;

 // Leer todas las líneas del archivo
 while (std::getline(file, line)) {
 // Eliminar caracteres de retorno de carro si existen
 line.erase(std::remove(line.begin(), line.end(), '\r'), line.end());
 if (!line.empty()) {
 lines.push_back(line);
 }
 }
 file.close();

 if (lines.empty()) {
 std::cerr << "Error: El archivo está vacío" << std::endl;
 return false;
 }

 // Determinar dimensiones del mapa
 m_height = lines.size();
 m_width = 0;
 for (const auto& l : lines) {
 m_width = std::max(m_width, (int)l.length());
 }

 // Inicializar la grid con nullptr
 m_grid.assign(m_height, std::vector<Tile*>(m_width, nullptr));

 // Procesar cada carácter
 for (int row = 0; row < m_height; ++row) {
 const std::string& currentLine = lines[row];
 for (int col = 0; col < m_width; ++col) {
 char tileChar = (col < currentLine.length()) ? currentLine[col] : ' ';
 ProcessTile(tileChar, col, row);
 }
 }
}

```

```

 }
}

std::cout << "Mapa cargado correctamente: " << m_width << "x" << m_height
 << ", " << m_tiles.size() << " tiles activos" << std::endl;

return true;
}

void MapReader::ProcessTile(char tileChar, int col, int row) {
 TileType type;

 // Convertir carácter a tipo de tile
 switch (tileChar) {
 case '#':
 type = TileType::FLOOR;
 break;
 case 'S':
 type = TileType::SPAWN;
 break;
 case 'E':
 type = TileType::ENEMY_SPAWN;
 break;
 case 'P':
 type = TileType::POWERUP;
 break;
 case 'O':
 type = TileType::OBJECTIVE;
 break;
 case '=':
 type = TileType::PLATFORM;
 break;
 case 'H':
 type = TileType::LADDER;
 break;
 case '^':
 type = TileType::SPIKE;
 break;
 case ' ':
 // Espacios vacíos no se añaden como tiles
 return;
 default:
 std::cerr << "Advertencia: Carácter desconocido " << tileChar
 << " en posición (" << col << ", " << row << ")" << std::endl;
 return;
 }

 // Crear y almacenar el tile
 Tile* tile = new Tile(type, col, row, tileChar);
 m_tiles.push_back(tile);
 m_grid[row][col] = tile;
}

std::vector<Tile*> MapReader::GetTilesByType(TileType type) const {

```

```

std::vector<Tile*> result;
for (Tile* tile : m_tiles) {
 if (tile->type == type) {
 result.push_back(tile);
 }
}
return result;
}

Tile* MapReader::GetPlayerSpawn() const {
 for (Tile* tile : m_tiles) {
 if (tile->type == TileType::SPAWN) {
 return tile;
 }
 }
 return nullptr;
}

std::vector<Tile*> MapReader::GetEnemySpawns() const {
 return GetTilesByType(TileType::ENEMY_SPAWN);
}

void MapReader::Clear() {
 for (Tile* tile : m_tiles) {
 delete tile;
 }
 m_tiles.clear();
 m_grid.clear();
 m_width = 0;
 m_height = 0;
}
...

```

### 3. Archivo de mapa de ejemplo: `level1.txt`

```

...
#####
#
S
#
#
#
P
#
O
#####
...

```

### 4. Ejemplo de uso en tu juego:

```

```cpp
#include "MapReader.h"
#include <SFML/Graphics.hpp>

```

```

int main() {
    MapReader map;

    if (!map.LoadFromFile("level1.txt")) {
        return -1;
    }

    // Crear spawn del jugador
    Tile* playerSpawn = map.GetPlayerSpawn();
    if (playerSpawn) {
        // Posición en píxeles (suponiendo tiles de 32x32)
        sf::Vector2f playerPosition(
            playerSpawn->x * 32.0f,
            playerSpawn->y * 32.0f
        );
        // Crear jugador aquí...
    }

    // Crear plataformas/pisos
    std::vector<Tile*> floors = map.GetTilesByType(TileType::FLOOR);
    for (Tile* floor : floors) {
        // Crear objeto Floor en floor->x, floor->y
    }

    // Crear power-ups
    std::vector<Tile*> powerups = map.GetTilesByType(TileType::POWERUP);
    for (Tile* powerup : powerups) {
        // Crear PowerUp en powerup->x, powerup->y
    }

    return 0;
}
...

```

5. Función para guardar el mapa (si necesitas edición):

```

...cpp
bool MapReader::SaveToFile(const std::string& filename) const {
    std::ofstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Error: No se pudo crear el archivo " << filename << std::endl;
        return false;
    }

    // Crear una grid de caracteres
    std::vector<std::vector<char>> charGrid(m_height, std::vector<char>(m_width, ' '));

    for (Tile* tile : m_tiles) {
        charGrid[tile->y][tile->x] = tile->originalChar;
    }

    // Escribir al archivo
    for (int row = 0; row < m_height; ++row) {
        for (int col = 0; col < m_width; ++col) {

```

```

        file << charGrid[row][col];
    }
    file << '\n';
}

file.close();
return true;
}
...
```

Ventajas de este enfoque:

1. ****Sin dependencias externas**** - No necesitas RapidXML ni ninguna librería
2. ****Fácil de editar**** - Cualquier editor de texto sirve
3. ****Visualmente claro**** - Puedes "ver" el mapa mientras lo editas
4. ****Sin problemas de compilación**** - No hay errores de iteradores ni includes
5. ****Rendimiento**** - Mucho más rápido que parsear XML
6. ****Simple**** - Menos código, menos errores

Caracteres sugeridos para tu juego:

- `#` - Suelo sólido / pared
- `S` - Spawn del jugador
- `E` - Spawn de enemigos
- `P` - Power-up / objeto coleccionable
- `O` - Objetivo / salida del nivel
- `=` - Plataforma (solo colisión desde arriba)
- `H` - Escalera
- `^` - Trampa / pincho
- `` - Espacio vacío

¿Necesitas que adapte algo específico para tu juego o que añada más funcionalidades?